

CLAUDE.md - Challenges Module

INHERITED FROM the Helix Constitution

This module is governed by the Helix Constitution. All rules in the constitution^{™s} CLAUDE.md and the Constitution.md it references apply unconditionally. Locate the constitution from any nested depth via its find_constitution.sh helper ^{â€} do NOT hardcode a path (this module stays fully decoupled and project-agnostic per ^Â§11.4.28).

Canonical reference: <https://github.com/HelixDevelopment/HelixConstitution>

Acceptance demo for this module

```
# Full challenge execution engine: Configure â†’ Execute â†’ Validate â†’  
cd Challenges && GOMAXPROCS=2 nice -n 19 go test -count=1 -race -v \  
-run 'TestRunner_Execute' ./...
```

Expect: PASS; a sample challenge runs to completion, assertion engine evaluates, and JSON/Markdown reports are emitted. The cmd/userflow-runner CLI (see Challenges/README.md) drives the same pipeline end-to-end.

Overview

digital.vasic.challenges is a generic, reusable Go module for defining, registering, executing, and reporting on challenges (structured test scenarios). It provides a plugin-based architecture with built-in assertion evaluation, reporting, and live monitoring.

Module: digital.vasic.challenges (Go 1.24+) **Depends on:**
digital.vasic.containers

Build & Test

```
go build ./...  
go test ./... -count=1 -race  
go test ./... -short           # Unit tests only  
go test -tags=integration ./... # Integration tests  
go test -bench=. ./tests/benchmark/
```

Code Style

- Standard Go conventions, gofmt formatting
- Imports grouped: stdlib, third-party, internal (blank line separated)
- Line length ^{â‰} 100 chars
- Naming: camelCase private, PascalCase exported, acronyms all-caps
- Errors: always check, wrap with fmt.Errorf("...: %w", err)
- Tests: table-driven, testify, naming Test<Struct>_<Method>_<Scenario>

Package Structure

Package	Purpose
pkg/challenge	Core types: Challenge interface, Config, Result, BaseChallenge
pkg/registry	Challenge registration, dependency ordering (Kahnâ€™s algo)
pkg/runner	Execution engine (sequential, parallel, pipeline)
pkg/assertion	Assertion engine with 16 built-in evaluators
pkg/report	Report generation (Markdown, JSON, HTML)
pkg/logging	Structured logging (JSON, Console, Multi, Redacting)
pkg/env	Environment variable handling with redaction
pkg/httpclient	Generic REST API client with JWT auth and functional options
pkg/bank	Challenge bank (load definitions from JSON/YAML)
pkg/monitor	Live monitoring with WebSocket dashboard
pkg/metrics	Prometheus-compatible challenge metrics
pkg/plugin	Plugin system for custom challenge types
pkg/infra	Infrastructure bridge to Containers module
pkg/userflow	Multi-platform user flow automation: adapters, templates, evaluators, container infra

Key Interfaces

- `challenge.Challenge` â€™ Challenge contract (ID, Configure, Validate, Execute, Cleanup)
- `registry.Registry` â€™ Challenge registration and dependency ordering
- `runner.Runner` â€™ Challenge execution (Run, RunAll, RunSequence, RunParallel)
- `assertion.Engine` â€™ Assertion evaluation with custom evaluators
- `report.Reporter` â€™ Report generation (Markdown, JSON, HTML)
- `logging.Logger` â€™ Structured logging with API request/response tracking
- `env.Loader` â€™ Environment variable management with redaction
- `plugin.Plugin` â€™ Plugin interface for extending the framework
- `infra.InfraProvider` â€™ Bridge to container infrastructure
- `userflow.BrowserAdapter` â€™ Browser automation (Playwright, Selenium, Cypress, Puppeteer)
- `userflow.MobileAdapter` â€™ Mobile automation (ADB, Appium, Maestro, Espresso)
- `userflow.DesktopAdapter` â€™ Desktop app automation (Tauri WebDriver)
- `userflow.APIAdapter` â€™ HTTP API testing adapter
- `userflow.GRPCAdapter` â€™ gRPC service testing (grpcurl CLI, unary + streaming)
- `userflow.WebSocketFlowAdapter` â€™ WebSocket flow testing (gorilla/websocket)
- `userflow.BuildAdapter` â€™ Build tool adapter (Gradle, Cargo, npm, Robolectric)
- `userflow.ProcessAdapter` â€™ Process lifecycle management
- `userflow.TestEnvironment` â€™ Container orchestration bridge to `digital.vasic.containers`

Design Patterns

- **Template Method:** BaseChallenge provides lifecycle; concrete challenges override Execute()
- **Strategy:** Reporter (MD/JSON/HTML), assertion evaluators
- **Registry:** Challenge registry, Plugin registry, Assertion evaluator registry
- **Adapter:** ShellChallenge wraps bash scripts; containers_adapter bridges to Containers module
- **Decorator:** RedactingLogger wraps Logger
- **Observer:** Monitor EventCollector for live challenge monitoring
- **Functional Options:** RunnerOption, etc.

Progress-Based Liveness Detection

Challenges may run for hours (e.g., scanning a 10TB NAS over SMB). Hard timeouts are **wrong** for long-running challenges. Instead, the framework uses progress-based liveness detection:

- **ProgressReporter** (pkg/challenge/progress.go): Buffered channel (64) for challenges to signal forward progress. Call ReportProgress(msg, data) periodically.
- **Liveness Monitor** (pkg/runner/liveness.go): Goroutine that watches the progress channel. If no progress is reported within StaleThreshold, the challenge is declared **stuck** and cancelled.
- **StatusStuck** ("stuck"): New terminal status distinct from "timed_out". Stuck = no progress; timed out = hard timeout exceeded.

Usage

```
// In your challenge's Execute():
func (c *MyChallenge) Execute(ctx context.Context) (*Result, error) {
    for i := range files {
        c.ReportProgress("scanning", map[string]any{
            "files_processed": i,
        })
        // ... do work ...
    }
}
```

The runner automatically attaches a ProgressReporter to any challenge that embeds BaseChallenge. Configure thresholds:

```
runner.NewRunner(
    runner.WithTimeout(72*time.Hour),           // Hard upper bound (generous)
    runner.WithStaleThreshold(5*time.Minute),    // Kill if no progress for 5 min
)
```

Per-challenge override via Config.StaleThreshold.

User Flow Automation (pkg/userflow)

Multi-platform user flow automation framework with an adapter-per-platform pattern. Generic “no project-specific references in pkg/userflow/.

Adapters (8 interfaces, 21 implementations): - BrowserAdapter â†’ PlaywrightCLIAdapter (CDP/WebSocket), SeleniumAdapter (W3C WebDriver), CypressCLIAdapter (Cypress CLI specs), PuppeteerAdapter (Node.js scripts) - MobileAdapter â†’ ADBCLIAdapter (Android/TV via adb), AppiumAdapter (Appium 2.0 W3C), MaestroCLIAdapter (Maestro YAML flows), EspressoAdapter (Gradle + ADB hybrid) - RecorderAdapter â†’ PanopticRecorderAdapter (CDP screencast), ADBRecorderAdapter (ADB screenrecord) - DesktopAdapter â†’ TauriCLIAdapter (Tauri apps via WebDriver) - APIAdapter â†’ HTTPAPIAdapter (REST API via pkg/httpclient) - GRPCAdapter â†’ GRPCCLIAdapter (gRPC via grpcurl CLI, unary + streaming) - WebSocketFlowAdapter â†’ GorillaWebSocketAdapter (gorilla/websocket, thread-safe) - BuildAdapter â†’ GradleCLIAdapter, CargoCLIAdapter, NPMCLIAdapter, RobolectricAdapter (Android JVM tests via Gradle) - ProcessAdapter â†’ SystemProcessAdapter

Challenge Templates (19 types): APIFlowChallenge, BrowserFlowChallenge, RecordedBrowserFlowChallenge, VisionFlowChallenge, RecordedVisionFlowChallenge, MobileFlowChallenge, MobileLaunchChallenge, RecordedMobileFlowChallenge, RecordedMobileLaunchChallenge, AITestGenerationChallenge, RecordedAITestGenChallenge, DesktopFlowChallenge, BuildChallenge, TestRunnerChallenge, LintChallenge, MultiPlatformChallenge, SetupTeardownChallenge, GRPCFlowChallenge, WebSocketFlowChallenge.

Recorded Challenge Templates wrap their non-recorded counterparts with RecorderAdapter, adding video recording with integrity verification (non-zero file size, duration, frame count). Use these for all UI challenges.

Container Integration: TestEnvironment bridges to digital.vasic.containers via PlatformGroup concept. Manages Podman container lifecycle (setup/teardown) per platform group within the 4 CPU / 8 GB resource budget.

CLI: cmd/userflow-runner â€” flags: --platform, --report, --compose, --root, --timeout, --output, --verbose.

Evaluators (12 userflow-specific): http_status_ok, http_status_created, http_status_unauthorized, http_status_forbidden, http_status_not_found, http_json_valid, browser_element_visible, browser_url_matches, mobile_activity_visible, mobile_element_exists, build_success, test_pass_rate.

Built-in Assertion Evaluators

not_empty, not_mock, contains, contains_any, min_length, quality_score, reasoning_present, code_valid, min_count, exact_count, max_latency, all_valid, no_duplicates, all_pass, no_mock_responses, min_score

Commit Style

Conventional Commits: feat(assertion): add custom evaluator support

Integration Seams

Direction	Sibling modules
Upstream (this module imports)	Containers
Downstream (these import this module)	HelixLLM, HelixQA, LLMsVerifier

Siblings means other project-owned modules at the project repo root. The root project app and external systems are not listed here – the list above is intentionally scoped to module-to-module seams, because drift *between* sibling modules is where the “tests pass, product broken” class of bug most often lives. See root CLAUDE .md for the rules that keep these seams contract-tested.